SECURE SOFTWARE DEVELOPMENT

Vulnerability Analysis and Remediation in a Java Student Portal

Techniques: Static Analysis • Symbolic Execution • Debugging

Course	Secure Software Development
Instructor	Dr. Yasser
Team members	1. _____ 2. _____ 3. _____ 4. _____ 5. _____ 6. _____ 7. _____
Student IDs	_____
Submission date	_____

1. Executive Summary

This project evaluates a small Java console application called Student Portal. Students can register an account, log in, and request a student profile by ID. The project deliberately includes three security defects in the training build and provides a second build with the corresponding remediations. The implementation uses object-oriented classes and Java LinkedList collections, and stores passwords as salted PBKDF2 hashes rather than plaintext.

The vulnerabilities are: (1) a hardcoded administrator credential, (2) an authentication logic flaw caused by an incorrect AND condition, and (3) missing object-level authorization when viewing a profile (IDOR/Broken Access Control). The techniques selected from the lecture slide are Static Analysis, Symbolic Execution, and Debugging. The slide lists testing and analysis techniques, not vulnerability categories.

2. Project Scope and Design

The application is intentionally small enough to inspect within an assignment timeframe. User accounts and student profiles are held in memory, so all data is cleared when the application exits. There is no database, network service, or external dependency.

Component	Responsibility	OOP / data-structure role
StudentPortalApp	Console menus, registration, login, profile lookup, session flow.	Application controller; owns two LinkedList collections.
UserAccount	Stores username and password-verification data.	Encapsulates account state; password data is private.
StudentProfile	Stores student ID, owner username, name, and department.	Encapsulates one profile and its ownership link.
PasswordHasher	Creates random salts and verifies PBKDF2 password hashes.	Centralizes password-hashing logic.

3. Deliberate Vulnerabilities

VULN-1 — Hardcoded administrator credential

Location: `authenticate()` in the vulnerable StudentPortalApp.java. The method compares a username and password against literal values embedded in source code and creates an administrator session. Anyone who can read the source can recover the credential; source control, copies, and screenshots can expose it as well.

```
if ("admin".equals(username) && "Admin@123".equals(password)) {
    return UserAccount.createAdminSession();
}
```

Remediation: remove the hardcoded credential and the shortcut session-creation path. Administrative identities in a production system must be provisioned through an appropriately protected configuration or identity-management process.

VULN-2 — Authentication logic flaw

Location: `authenticate()` in the vulnerable build. The account is first retrieved by the supplied username. The following condition uses AND to reject a login only if both the username comparison and password comparison fail. Because the retrieved account already matches the supplied username, the first negated comparison is false, and the rejection branch is skipped even when the password is wrong.

```
if (!account.getUsername().equals(username) && !account.passwordMatches(password)) {
    return null;
}
```

Remediation: reject the request when the account does not exist OR the password does not match.

```
if (account == null || !account.passwordMatches(password)) {
    return null;
}
```

VULN-3 — IDOR / Broken Access Control

Location: `viewStudentProfile()` in the vulnerable build. The application looks up a profile by student ID and displays it without comparing the current account with the profile owner. A user can therefore request another user's predictable or otherwise discovered ID.

```
StudentProfile profile = findStudentById(studentId);
if (profile == null) {
    System.out.println("Student profile not found.");
    return;
}
System.out.println(profile.toDisplayString());
```

Remediation: enforce authorization for the requested object before returning its fields.

```
if (!profile.getOwnerUsername().equals(currentUser.getUsername())) {
    System.out.println("Access denied. You can only view your own profile.");
    return;
}
```

4. Techniques Applied

4.1 Static Analysis

Static analysis examines source code without executing the application. For this project, the reviewer inspects the authentication method, searches for credential-like string literals, checks boolean conditions, and verifies whether a sensitive data-return path checks ownership. An IDE inspection or a tool such as SonarLint/SpotBugs may assist, but the report does not claim an external analyzer was run.

- Finding: the administrator credential appears as a literal in the vulnerable source.
- Finding: the login rejection condition combines negated checks with AND in a way that permits an invalid password.
- Finding: profile data is returned without a check tying the requested profile to the authenticated user.

4.2 Symbolic Execution

Symbolic execution reasons about program paths using logical conditions. The authentication condition can be represented using A and B:

Symbol	Expression	Value for an existing username and wrong password
A	NOT (account.username == username)	false; findUser selected this same username
B	NOT passwordMatches(password)	true; the password is wrong
A AND B	Rejection condition	false AND true = false; rejection branch is skipped

This is a manual symbolic/path-condition analysis of the source condition. It demonstrates the technique's reasoning, but it is not a claim that a dedicated symbolic-execution engine was run.

4.3 Debugging

Set a breakpoint after `findStudentById(studentId)` in `viewStudentProfile()`. Log in as Alice and request Bob's profile ID. Inspect `currentUser.getUsername()`, `profile.getOwnerUsername()`, and the next statement. In the vulnerable build, these usernames differ, yet execution proceeds to display the profile. In the secure build, the ownership check returns before any profile fields are shown.

5. Manual Verification Plan and Results

Test	Procedure	Vulnerable build	Secure build
TC-01: embedded credential	Try admin / Admin@123.	Login succeeds as ADMIN.	Login fails; no built-in administrator shortcut.
TC-02: wrong password	Register alice, then log in as alice with wrong-password.	Login succeeds because the rejection condition is flawed.	Login is rejected.
TC-03: cross-user profile	Register Alice and Bob; log in as Alice and request Bob's ID.	Bob's profile is displayed.	Request is rejected with Access denied.
TC-04: own profile	Log in correctly and request own ID.	Own profile is displayed.	Own profile is displayed.
TC-05: weak password	Try registering with a simple/short password.	Registration is rejected by the policy.	Registration is rejected by the policy.

The supplied source builds were compiled with a Java JDK and exercised using repeatable console input sequences for the three planned defects. For a final presentation, capture screenshots showing each vulnerable line, the vulnerable output, the corrected line, and the secure output. See TEST_PLAN.md for the exact keystrokes.

6. Vulnerable vs. Secure Build

Control	Vulnerable build	Secure build
Administrator login	Hardcoded username/password branch creates an ADMIN session.	No built-in admin credential or shortcut session.
Password verification	Incorrect AND condition bypasses password verification for an existing username.	Rejects if account is absent OR password verification fails.
Profile authorization	Returns any profile found by ID.	Compares profile owner to current username before returning data.
Password storage	Random salt + PBKDF2 hash for registered users.	Same PBKDF2 hash approach; no plaintext account password storage.
Data structure	LinkedList of users and profiles.	LinkedList of users and profiles.

7. Team Work Distribution

Replace the placeholders below with the actual members and the work each person completed. A four-person example aligns one developer with the three selected techniques.

Member / ID	Suggested responsibility	Evidence / deliverable
Member 1: _____	Implement the Java application and compare vulnerable vs. secure builds.	Source code and comparison section.
Member 2: _____	Apply Static Analysis to the source.	Screenshot of hardcoded credential and suspicious conditions.
Member 3: _____	Apply Symbolic Execution reasoning to the login condition.	Path-condition table and explanation.
Member 4: _____	Use Debugging to trace the profile-authorization path.	Breakpoint variables and access-control test.

8. Limitations and Conclusion

This is a local educational prototype, not a production-ready student information system. Accounts and profiles exist only for the current process; there is no database, encrypted transport, persistent session management, audit logging, or production identity provisioning. The purpose is to demonstrate three controllable defects, show how selected analysis techniques reveal them, and compare a vulnerable build with a corrected build.

The main lesson is that authentication and authorization are separate controls: verifying a user's identity is not enough; the application must also verify that the user is permitted to access the particular object they requested. Hardcoded secrets and incorrect Boolean logic can undermine authentication even in a small program.

Appendix A. Run Commands

```
cd src-vulnerable
javac *.java
java StudentPortalApp

cd src-secure
javac *.java
java StudentPortalApp
```

Compile each directory independently because both builds contain a class named StudentPortalApp. No third-party libraries are required.